



Faculty of Engineering **Cairo University**

Medical Pattern Recognition

Task1

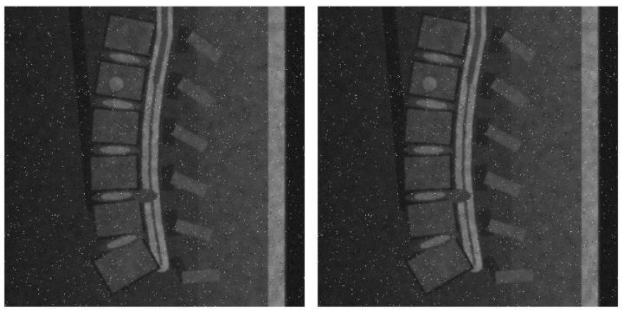
Submitted by:

Mennat Allah Nabil 1220105

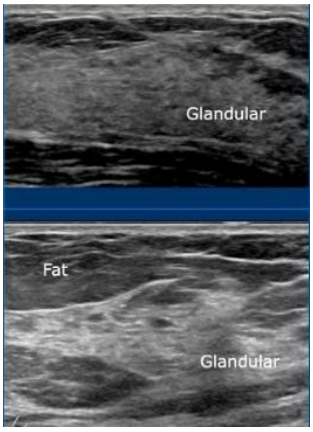
Image Generation :

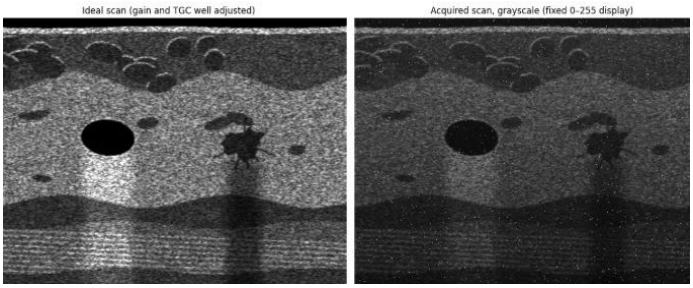
In the image-generation stage, I considered several ideas for creating a suitable synthetic medical image using Python only. At first, I search MRI slices and all the results were barin images, but it felt way too complicated to try. So I went for simpler images such as: spine, knee and even tried ultrasound were explored. However, some of these options either looked too simplified or required drawing details that made the code more complex than needed. After trying different directions, I chose blood smear as the final image source. It is closely related to pattern recognition through cell shape, size, and intensity variation, and it can be generated in a clear and understandable way using basic Python drawing functions. The final generated image included red blood cells, white blood cells, and mild noise, which made it appropriate for the preprocessing tasks in the experiment such as grayscale conversion, histogram analysis, histogram equalization, CLAHE, and spatial filtering.

Below are trials of my image generations:



1- The Spine was my second go to option after the blood smear but I felt the contrast levels may not present the histogram and CDF the way blood smear did.
2- On the left, this was the reference image I searched for when looking into ultrasounds. The breast ultrasound image, but as we can see the details are too complex and I didn't find my trial below satisfying in reaching the reference image.

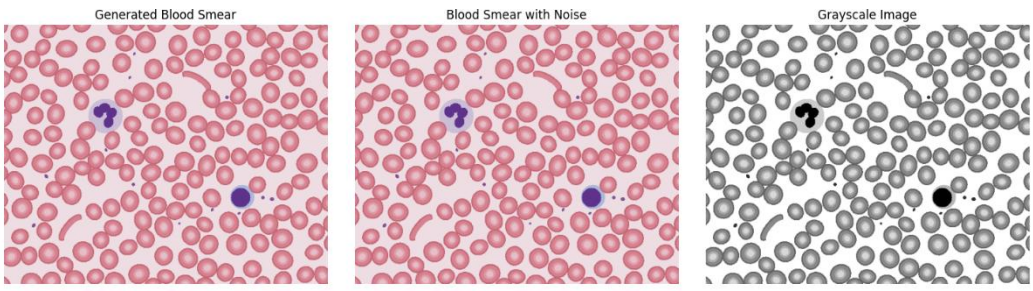




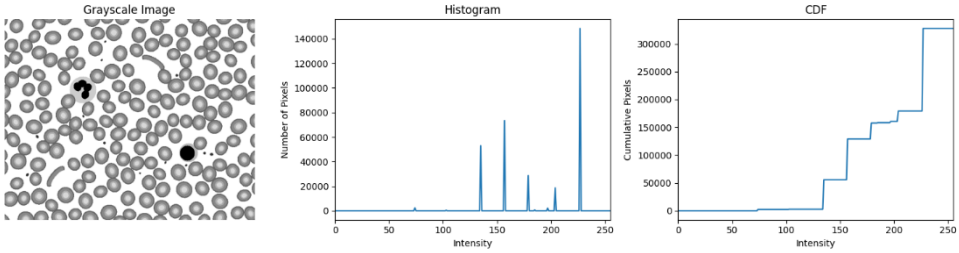
3- Third try was the knee. As we can see, total failure in resemblance. No explanation needed in why I didn't continue working on this one :D.



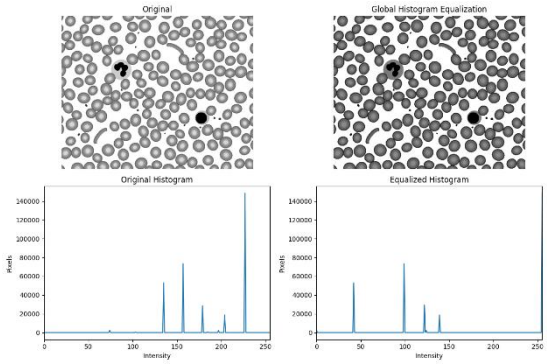
Finally, I thought of microscopic images. The blood smear was accurate compared to the real image. Also I think it did a great job capturing all the filters and histogram effects.



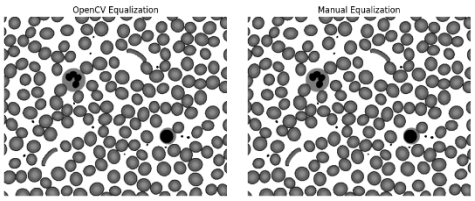
CDF Check:



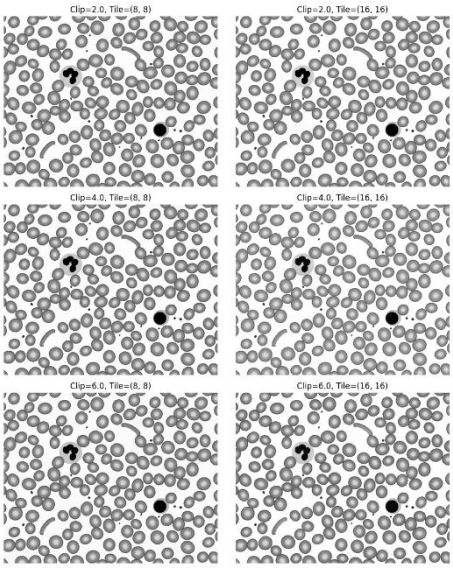
I checked the CDF graph and I still found no notes of obvious fault. It increases cumulatively with intensity. The image contains lots of pixels with similar intensities, so the cumulative sum stays almost flat for a while then jumps sharply whenever it reaches one of those heavily populated intensity values. Its final value also equals the total number of pixels in the image, confirming that it was calculated correctly from the histogram.



Global histogram equalization: Global equalization spread the intensity values over a wider range, making differences between the cells and background more visible. However, because one transformation is applied to the whole image, noise can also become more noticeable.

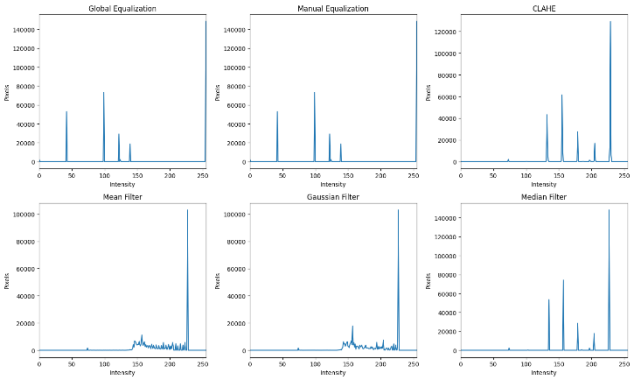


Manual histogram equalization: The CDF-based equation was implemented manually using a lookup table. The manual output was consistent with cv2.equalizeHist(), showing that the manual calculation was performed correctly.



CLAHE: Different clip limits and tile sizes were tested. CLAHE improved contrast in local parts of the image rather than treating the whole image in the same way. Increasing the clip limit generally produced stronger enhancement but could also make noise more visible.

Spatial filtering: The mean and Gaussian filters both smoothed the image and reduced random noise. The mean filter produced more general blurring, while Gaussian filtering gave smoother results with more emphasis on nearby pixels. The median filter was particularly effective at removing the added salt-and-pepper pixels while preserving cell boundaries better.



Quantitative comparison: The minimum, maximum, mean, standard deviation, and histograms were compared for the six preprocessing outputs. The equalization methods produced a wider intensity distribution, while the filtering methods reduced intensity variation because of their smoothing effect. Global and manual equalization produced very similar results.